

Aula 3 - Vetores (Parte 2)

Os arrays, embora extremamente úteis do jeito que são, têm uma grande limitação: o tamanho fixo. Ao declarar um array com n elementos, ele terá n elementos até o fim da execução do programa. Em alguns casos, isso não é interessante.

Aí, entra a primeira biblioteca que estudaremos: `vector`.

Até então, o template básico de um programa em C++ que estávamos usando era o seguinte:

```
#include <iostream>
using namespace std;

int main(){
    // código inteiro aqui
}
```

Agora, precisaremos de mais uma linha de include:

```
#include <iostream>
#include <vector> // linha nova!
using namespace std;

int main(){
    // código inteiro aqui
}
```

Esses `#include` dizem respeito à inclusão de **bibliotecas**. As bibliotecas são conjuntos de funções e/ou estruturas de dados que podem ser úteis para um programa.

Até o momento, só usávamos a `iostream`, que continha as funções básicas de input/output do C++ (`cin`, `cout` ...). A biblioteca `vector` inclui uma nova estrutura de dados: (*passagem*) o **vector**.

Os vectors são vetores (tal qual os arrays), isso significa que eles dizem respeito à uma sequência de espaços reservados na memória do seu computador. Ou seja, na prática, são um monte de variáveis convenientes.

A grande vantagem do uso de vectors é o seu tamanho variável, podemos aumentar ou diminuir seu tamanho livremente, bem como temos acesso a algumas funções extras que são úteis em alguns momentos. Sua sintaxe é bem similar à de arrays, com algumas diferenças:

```
vector<int> meu_vetor; // essa linha declara a existência de
um vector de inteiros
int meu_array[10]; // é equivalente a essa linha
```

Note que no caso do vector, nós não temos a obrigação de especificar o tamanho do vetor na sua criação. Basta o tipo de variável que guardaremos nele (entre os <>) e o seu nome. Se tentarmos acessar o meu_vetor logo depois de criarmos ele, teremos um problema, ele está vazio (e, portanto, tem tamanho 0). Acessaremos lixo de memória. Então, precisamos adicionar elementos nele:

```
// inicialmente: meu_vetor = {}
meu_vetor.push_back(99);
// agora: meu_vetor = {99}
meu_vetor.push_back(102);
// meu_vetor = {99, 102}
meu_vetor.push_back(200);
// meu_vetor = {99, 102, 200}
```

A função `push_back(x)` coloca, no final do vetor, o elemento x . A cada vez que um elemento é inserido, o tamanho do vetor aumenta.

Além da função de inserção, temos também uma função de remoção (que é absurdamente menos utilizada em programação competitiva, eu particularmente nunca usei ela):

```
meu_vetor.pop_back(); // exclui o último elemento do vetor
// note que não precisamos colocar nada dentro do parêntese
```

Infelizmente, em um vetor, nós só conseguimos nos livrar do último elemento. Excluir um elemento do meio do vetor é impossível (mas isso quase nunca é necessário de qualquer forma).

Além de inserir e remover elementos de um vetor, conseguimos também consultar seu tamanho atual:

```
cout << meu_vetor.size(); // vai imprimir um inteiro equivalente ao tamanho do meu_vetor
```

Obs.: essa função `.size()` também funciona para strings (e é bem útil).

Por fim, lembramos que, ao declarar um vector, não temos a obrigação de especificar seu tamanho:

```
vector<int> meu_vetor2; // funciona perfeitamente
```

Mas, se quisermos por algum motivo declarar um vector com um tamanho previamente definido (tal qual fazemos nos arrays) nós podemos. Isso não interfere na capacidade do vector de aumentar e diminuir de tamanho, apenas ditamos o seu tamanho inicial:

```
vector<int> meu_vetor2(20); // isso diz que meu_vetor2 tem 20 posições, de 0 a 19
vector<int> meu_vetor3(n); // isso diz que meu_vetor3 tem n posições
// pra que esse último funcione, n tem que ser uma variável declarada (e com valor)
```

Adicionalmente, se quisermos, além de declarar um tamanho, preencher previamente um vector com um valor padrão inicial, nós conseguimos também:

```
vector<int> tudo_zero(40, 0); // isso diz que o tudo_zero tem 40 posições
// e em todas elas, temos um 0

// funciona da mesma forma para outros tipos de variáveis
```

```
vector<bool> tudo_falso(100, false);  
vector<char> tudo_g(n, 'g');
```

Resumo de tudo

Para declarar um vector, temos duas opções:

```
vector<tipo> nome_do_vetor;  
vector<tipo> nome_do_vetor(tamanho, valor_padrao);
```

Temos 3 funções principais:

```
nome_do_vetor.push_back(valor);  
nome_do_vetor.pop_back();  
nome_do_vetor.size();
```

E consultamos/alteramos informações dentro do vetor da mesma maneira que nos arrays:

```
nome_do_vetor[posicao]
```